

# 尚硅谷嵌入式技术之 Modbus 协议

（作者：尚硅谷研究院）

版本：V1.0

## 第 1 章 协议背景

### 1.1 Modbus 简介

Modbus 是一种广泛应用于工业自动化领域的串行通信协议，由 Modicon 公司（现施耐德电气）于 1979 年推出，主要用于可编程逻辑控制器（PLC）与其他设备之间的通信。其设计目标是简单、开放和高效，支持多种传输方式（如 RS-232/485、以太网等），已成为工业控制系统的行业标准之一。

### 1.2 Modbus 的核心特点

#### 1) 开放协议

Modbus 协议规范公开免费，无需授权即可使用，促进了其广泛普及。

#### 2) 简单性

协议结构简洁，数据帧格式易于实现和调试，适合资源有限的嵌入式设备。

#### 3) 广泛应用

支持多种物理层（RS-485、RS-232、以太网等）和传输方式（RTU、ASCII、TCP）。

#### 4) 主从架构

通信基于主设备（Master）发起请求，从设备（Slave）响应，支持单主或多从结构。

#### 5) 数据模型

定义四种数据类型（线圈、离散输入、保持寄存器、输入寄存器），通过地址直接访问设备数据。

### 1.3 Modbus 的协议版本

#### 1) Modbus RTU (Remote Terminal Unit)

传输方式：二进制编码，通过串行通信（RS-485/RS-232）传输。

帧格式：

设备地址（1 字节） + 功能码（1 字节） + 数据（N 字节） + CRC 校验（2 字节）。

特点：高效紧凑，适合工业现场环境，是应用最广泛的版本。

## 2) Modbus ASCII

传输方式：ASCII 字符编码，每个字节以十六进制字符表示。

帧格式：

起始符 : + 设备地址（2 字符） + 功能码（2 字符） + 数据 + LRC 校验（2 字符） + 结束符 \r\n。

特点：可读性强，但效率低于 RTU，适用于调试场景。

## 3) Modbus TCP

传输方式：基于 TCP/IP 协议，通过以太网传输。

帧格式：

MBAP 头（7 字节，含事务标识、协议标识、长度、单元标识） + 功能码（1 字节） + 数据。

特点：支持高速网络通信，常用于现代工业以太网系统，默认端口 502。

# 第 2 章 功能码

## 2.1 主从发送

Modbus RTU 通信基于主从架构，主站（Master）发送请求帧，从站（Slave）返回响应帧。

帧格式如下：

设备地址（1 字节） + 功能码（1 字节） + 数据（N 字节） + CRC 校验（2 字节）

其中设备地址用于多个从设备识别，具体方法需要从设备接收数据帧时判断。

功能码代表不同的命令。

数据表示写入的内容，不同的功能码写入的内容不同。

CRC 校验用于收发消息确认是否有效。

## 2.2 常用功能码

01 (0x01) - 读取线圈状态（Read Coil Status）：用于读取一个或多个输入或输出（DO/DI）的状态。

02 (0x02) - 读取输入状态 (Read Input Status)：用于读取一个或多个离散输入 (DI) 的状态。

03 (0x03) - 读取保持寄存器 (Read Holding Registers)：用于读取一个或多个保持寄存器 (通常是模拟量输出, AO) 的内容。

04 (0x04) - 读取输入寄存器 (Read Input Registers)：用于读取一个或多个输入寄存器 (通常是模拟量输入, AI) 的内容。

05 (0x05) - 写入单个线圈 (Write Single Coil)：用于设置一个离散输出 (DO) 的状态。

06 (0x06) - 写入单个寄存器 (Write Single Register)：用于设置一个保持寄存器 (通常用于模拟量输出, AO) 的值。

15 (0x0F) - 写入多个线圈 (Write Multiple Coils)：用于同时设置多个离散输出 (DO) 的状态。

16 (0x10) - 写入多个寄存器 (Write Multiple Registers)：用于同时设置多个保持寄存器 (通常用于模拟量输出, AO) 的值。

| 功能码 | 十六进制表示 | 描述                                 |
|-----|--------|------------------------------------|
| 01  | 0x01   | 读取线圈状态 (Read Coil Status)          |
| 02  | 0x02   | 读取输入状态 (Read Input Status)         |
| 03  | 0x03   | 读取保持寄存器 (Read Holding Registers)   |
| 04  | 0x04   | 读取输入寄存器 (Read Input Registers)     |
| 05  | 0x05   | 写入单个线圈 (Write Single Coil)         |
| 06  | 0x06   | 写入单个寄存器 (Write Single Register)    |
| 15  | 0x0F   | 写入多个线圈 (Write Multiple Coils)      |
| 16  | 0x10   | 写入多个寄存器 (Write Multiple Registers) |

## 第 3 章 RTU 通信时序

### 3.1 波特率和字符时间

每个“字符时间”取决于通信的波特率。例如, 在 9600 波特率下, 一个比特时间为 1/9600 秒, 考虑到标准的 8 个数据位、1 个起始位、1 个停止位和无校验位 (共 11 位) 的情况, 一个字符的时间大约是 1.146 毫秒。所以, 3.5 个字符时间大约是 4 毫秒。

### 3.2 字符间时间间隔

这是指在一个 Modbus RTU 消息帧内，两个连续字符之间的最大允许时间间隔。如果这个时间间隔超过了规定的值（通常为 3.5 个字符时间），则认为当前消息帧已经结束。这意味着接收设备会将超过此间隔的任何后续字符视为新消息的开始。

字符时间和串口的波特率有关，常见的波特率有 9600，19200 和 115200。如果使用低速波特率（小于 19200），要严格按照 3.5 个字符时间，如果使用高速波特率，可以直接使用 35 个 50us（1.75ms）作为标准时间。

## 3.3 错误检测和超时

Modbus RTU 使用 CRC（循环冗余校验）进行错误检测。如果接收方检测到 CRC 不匹配，则该消息会被丢弃，等待下一个有效消息。

## 第 4 章 数据格式



Modbus通信协议  
4.0.xlsx

## 第 5 章 freeModbus 移植

### 5.1 下载源码

FreeModbus 的源码可以直接到对应的 github 下载，也可以直接使用资料中已经下载完成的源码。

<https://github.com/cwaller-at/freemodbus>

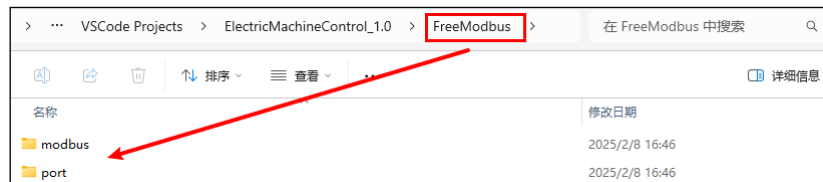
### 5.2 源码目录

|                                     |            |                          |             |
|-------------------------------------|------------|--------------------------|-------------|
| cwaller-at - Updated licensing info |            | f167010 · 7 years ago    | 2 Commits   |
| demo                                | 移植案例       | - Initial import         | 7 years ago |
| doc                                 |            | - Updated licensing info | 7 years ago |
| modbus                              | modbus源码本身 | - Updated licensing info | 7 years ago |
| tools                               |            | - Initial import         | 7 years ago |
| Changelog.txt                       |            | - Updated licensing info | 7 years ago |
| bsd.txt                             |            | - Initial import         | 7 years ago |
| gpl.txt                             |            | - Initial import         | 7 years ago |
| lgpl.txt                            |            | - Initial import         | 7 years ago |

其中 modbus 目录存放的是该库的可信源代码，demo 库中存放了一些移植案例。

(1) 在项目文件夹中新建 Freemodbus 目录，将 modbus 目录拷贝到该目录下（其中包含 TCP 协议内容，可加可不加）；

(2) demo 中有很多移植案例。这里我们是裸机移植，找到 BARE 文件夹，将其中的 port 拷贝到 Freemodbus 目录下面。最终目录结构如下图所示：

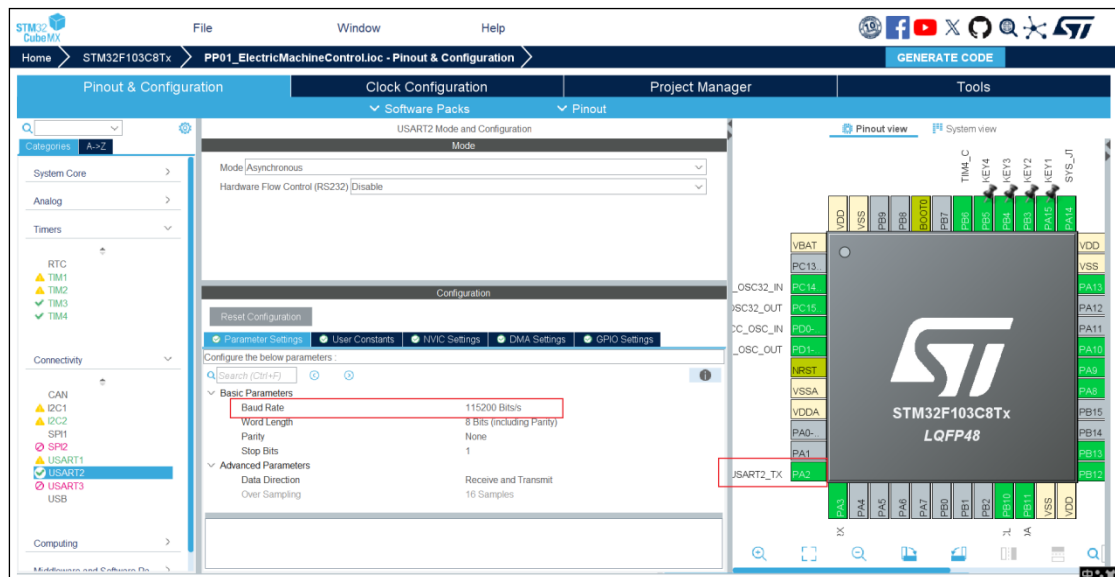


移植工作需要修改 port\_serial.c、port\_timer.c、新建 port.c 实现回调函数、修改 mb.h 方便修改缓存。

## 5.3 串口移植

Modbus 底层需要使用串口协议来完成基础的传输，这里直接使用 HAL 库配置的 USART2 来实现。

使用 CubeMax 开启 USART2 功能，配置波特率为 115200。并且打开中断，可以不使用 hal 生成中断函数。



修改 portserial.c 文件，适配串口 2。

### 5.3.1 portserial.c

```
/*
 * FreeModbus Library: BARE Port
```

```
* Copyright (C) 2006 Christian Walter <wolti@sil.at>
*
* This library is free software; you can redistribute it and/or
* modify it under the terms of the GNU Lesser General Public
* License as published by the Free Software Foundation; either
* version 2.1 of the License, or (at your option) any later version.
*
* This library is distributed in the hope that it will be useful,
* but WITHOUT ANY WARRANTY; without even the implied warranty of
* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
* Lesser General Public License for more details.
*
* You should have received a copy of the GNU Lesser General Public
* License along with this library; if not, write to the Free Software
* Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-
1301 USA
*
* File: $Id$
*/

#include "port.h"

/* ----- Modbus includes -----
-----*/
#include "mb.h"
#include "mbport.h"

/* ----- static functions -----
-----*/
static void prvvUARTTxReadyISR(void);
static void prvvUARTRxISR(void);

/* ----- Start implementation -----
-----*/
```

```
void vMBPortSerialEnable(BOOL xRxEnable, BOOL xTxEnable)
{
    /* If xRXEnable enable serial receive interrupts. If xTxENable
enable
    * transmitter empty interrupts.
    */
    // 开启发送和接收中断
    if (xRxEnable)
    {
        __HAL_UART_ENABLE_IT(&huart2, UART_IT_RXNE); // 使能接收中断
    }
    else
    {
        __HAL_UART_DISABLE_IT(&huart2, UART_IT_RXNE);
    }

    if (xTxEnable)
    {
        __HAL_UART_ENABLE_IT(&huart2, UART_IT_TXE); // 使能发送为空中断
    }
    else
    {
        __HAL_UART_DISABLE_IT(&huart2, UART_IT_TXE); // 使能发送为空中断
    }
}

BOOL xMBPortSerialInit(UCHAR ucPORT, ULONG ulBaudRate, UCHAR
ucDataBits, eMBParity eParity)
{
    // 直接使用 HAL 库初始化
    // MX_USART2_UART_Init();
    return TRUE;
}
```

```
BOOL xMBPortSerialPutByte(CHAR ucByte)
{
    /* Put a byte in the UARTs transmit buffer. This function is called
     * by the protocol stack if pxMBFrameCBTransmitterEmpty( ) has been
     * called. */
    USART2->DR = ucByte;
    return TRUE;
}

BOOL xMBPortSerialGetByte(CHAR *pucByte)
{
    /* Return the byte in the UARTs receive buffer. This function is
    called
     * by the protocol stack after pxMBFrameCBByteReceived( ) has been
    called.
     */
    *pucByte = (USART2->DR & (uint16_t)0x00FF);
    return TRUE;
}

/* Create an interrupt handler for the transmit buffer empty interrupt
 * (or an equivalent) for your target processor. This function should
 then
 * call pxMBFrameCBTransmitterEmpty( ) which tells the protocol stack
 that
 * a new character can be sent. The protocol stack will then call
 * xMBPortSerialPutByte( ) to send the character.
 */
static void prvvUARTTxReadyISR(void)
{
    pxMBFrameCBTransmitterEmpty();
}
```



```
/* Create an interrupt handler for the receive interrupt for your
target
* processor. This function should then call
pxMBFrameCByteReceived( ). The
* protocol stack will then call xMBPortSerialGetByte( ) to retrieve
the
* character.
*/
static void prvUARTRxISR(void)
{
    pxMBFrameCByteReceived();
}

// 编写回调
void USART2_IRQHandler(void)
{
    // 检查接收数据寄存器非空标志位是否被置位
    if ( __HAL_UART_GET_FLAG(&huart2, UART_FLAG_RXNE)) // 接收非空中断标记
        被置位
    {
        // 清除接收数据寄存器非空标志位
        __HAL_UART_CLEAR_FLAG(&huart2, UART_FLAG_RXNE); // 清除中断标记
        // 通知 modbus 有数据到达
        prvUARTRxISR(); // 通知 modbus 有数据到达
    }

    // 检查发送数据寄存器为空标志位是否被置位
    if ( __HAL_UART_GET_FLAG(&huart2, UART_FLAG_TXE)) // 发送为空中断标记
        被置位
    {
        // 清除发送数据寄存器为空标志位
        __HAL_UART_CLEAR_FLAG(&huart2, UART_FLAG_TXE); // 清除中断标记
        // 通知 modbus 数据可以发送
        prvUARTTxReadyISR(); // 通知 modbus 数据可以发送
    }
}
```

```

}

// 其他中断交给 hal 库处理

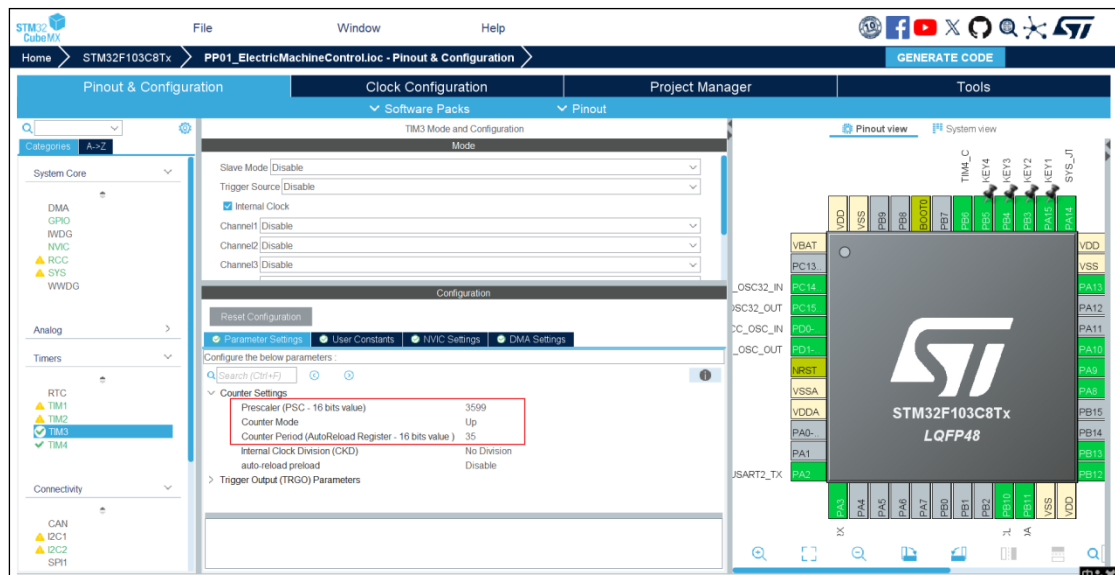
HAL_UART_IRQHandler(&huart2);
}

```

## 5.4 定时器移植

根据 Modbus 的时序可知，需要使用一个定时器判断传输数据是否超过 3.5 字符的时间，这里使用 TIM3。官方推荐定时器单次计数时间为 50us，所以 PSC 为 3599，串口波特率大于 19200 时，可以直接使用固定 35 次来代替 3.5 字符的时间。

根据要求使用 CubeMax 配置 TIM3：



### 5.4.1 porttimer.c

```

/*
 * FreeModbus Library: BARE Port
 * Copyright (C) 2006 Christian Walter <wolti@sil.at>
 *
 * This library is free software; you can redistribute it and/or
 * modify it under the terms of the GNU Lesser General Public
 * License as published by the Free Software Foundation; either
 * version 2.1 of the License, or (at your option) any later version.
 *
 * This library is distributed in the hope that it will be useful,

```

```
* but WITHOUT ANY WARRANTY; without even the implied warranty of
* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
* Lesser General Public License for more details.
*
* You should have received a copy of the GNU Lesser General Public
* License along with this library; if not, write to the Free Software
* Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-
1301 USA
*
* File: $Id$
*/

/* ----- Platform includes -----
-----*/
#include "port.h"

/* ----- Modbus includes -----
-----*/
#include "mb.h"
#include "mbport.h"

/* ----- static functions -----
-----*/
static void prvTIMERExpiredISR(void);

/* ----- Start implementation -----
-----*/
BOOL xMBPortTimersInit(USHORT usTim1Timerout50us)
{
    // 初始化定时器 3 要求 50us 一次计数
    // MX_TIM3_Init();
    // 单独开启计数溢出中断
    __HAL_TIM_CLEAR_FLAG(&htim3, TIM_FLAG_UPDATE); // 先清除一下定时器的
    中断标记,防止使能中断后直接触发中断
}
```

```
    __HAL_TIM_ENABLE_IT(&htim3, TIM_IT_UPDATE);    // 使能定时器更新中断
    return TRUE;
}

void vMBPortTimersEnable()
{
    /* Enable the timer with the timeout passed to xMBPortTimersInit( )
    */
    // 重置计数值 => 启动定时器 3
    __HAL_TIM_SET_COUNTER(&htim3, 0);
    HAL_TIM_Base_Start_IT(&htim3);
}

void vMBPortTimersDisable()
{
    /* Disable any pending timers. */
    // 关闭定时器 3
    HAL_TIM_Base_Stop_IT(&htim3);
}

/* Create an ISR which is called whenever the timer has expired. This
function
* must then call pxMBPortCBTimerExpired( ) to notify the protocol
stack that
* the timer has expired.
*/
static void prvvTIMERExpiredISR(void)
{
    (void)pxMBPortCBTimerExpired();
}

// 定时器溢出回调回调函数 => APP_RUN.C
// 或者单独启动 TIM3 的中断处理
void TIM3_IRQHandler(void)
```

```
{
    if (__HAL_TIM_GET_FLAG(&htim3, TIM_FLAG_UPDATE)) // 更新中断标记被置
位
    {
        __HAL_TIM_CLEAR_FLAG(&htim3, TIM_FLAG_UPDATE); // 清除中断标记
        prvvtIMERExpiredISR();                          // 通知 modbus3.5
个字符等待时间到
    }
}
```

## 5.5 从机接收命令回调移植

创建文件 port.c 用于实现从机接收命令的回调函数。

### 5.5.1 port.c

```
#include "mb.h"
#include "mbport.h"

// 声明输入寄存器缓冲区，用于存储十路输入寄存器的值
uint16_t REG_INPUT_BUF[REG_INPUT_SIZE];

// 声明保持寄存器缓冲区，用于存储十路保持寄存器的值
uint16_t REG_HOLD_BUF[REG_HOLD_SIZE];

// 定义十路线圈的大小
uint8_t REG_COILS_BUF[REG_COILS_SIZE] = {1, 1, 1, 1, 0, 0, 0, 0, 1, 1};

// 声明离散量缓冲区，并初始化，用于存储十路离散量的状态
uint8_t REG_DISC_BUF[REG_DISC_SIZE] = {1,1,1,1,0,0,0,0,1,1};

/**
 * @brief CMD4 命令处理回调函数
 *
 * 该函数用于处理 MODBUS 协议中的 CMD4 命令，即读取输入寄存器。
```

```
* 它将指定地址范围内的输入寄存器的值复制到缓冲区中。
*
* @param pucRegBuffer 指向用于存储寄存器值的缓冲区的指针。
* @param usAddress 要读取的起始寄存器地址。
* @param usNRegs 要读取的寄存器数量。
*
* @return 返回执行结果的错误代码。
*/
eMBCError eMBRegInputCB(UCHAR *pucRegBuffer, USHORT usAddress,
USHORT usNRegs)
{
    // 计算寄存器索引，从 0 开始
    USHORT usRegIndex = usAddress - 1;

    // 非法检测：检查访问范围是否超出寄存器缓冲区大小
    if ((usRegIndex + usNRegs) > REG_INPUT_SIZE)
    {
        return MB_ENOREG;
    }

    // 循环读取寄存器值并写入缓冲区
    while (usNRegs > 0)
    {
        // 将寄存器的高 8 位写入缓冲区
        *pucRegBuffer++ = (unsigned char)(REG_INPUT_BUF[usRegIndex] >>
8);

        // 将寄存器的低 8 位写入缓冲区
        *pucRegBuffer++ = (unsigned char)(REG_INPUT_BUF[usRegIndex] &
0xFF);

        usRegIndex++;
        usNRegs--;
    }

    return MB_ENOERR;
}
```

```
}

/**
 * @brief CMD6、3、16 命令处理回调函数
 *
 * 该函数用于处理 Modbus 协议中的 Holding Registers 读写请求。
 * 它根据请求的模式（读或写）对指定的寄存器进行相应的操作。
 *
 * @param pucRegBuffer 寄存器数据缓冲区，用于读取或写入寄存器数据。
 * @param usAddress 请求访问的起始寄存器地址。
 * @param usNRegs 请求访问的寄存器数量。
 * @param eMode 访问模式，可以是 MB_REG_WRITE（写寄存器）或 MB_REG_READ（读寄存器）。
 *
 * @return 返回执行结果，如果成功则返回 MB_ENOERR，否则返回相应的错误代码。
 */
eMBCErrorCode eMBRegHoldingCB(UCHAR *pucRegBuffer, USHORT usAddress,
USHORT usNRegs, eMBRegisterMode eMode)
{
    // 计算寄存器索引，Modbus 地址从 1 开始，数组索引从 0 开始，因此需要减 1。
    USHORT usRegIndex = usAddress - 1;

    // 非法检测：检查访问范围是否超出寄存器缓冲区大小。
    if ((usRegIndex + usNRegs) > REG_HOLD_SIZE)
    {
        return MB_ENOREG;
    }

    // 写寄存器
    if (eMode == MB_REG_WRITE)
    {
        // 循环将每个寄存器的数据从缓冲区写入到寄存器中。
        while (usNRegs > 0)
```

```
{
    REG_HOLD_BUF[usRegIndex] = (pucRegBuffer[0] << 8) |
pucRegBuffer[1];
    pucRegBuffer += 2;
    usRegIndex++;
    usNRegs--;
}
}
// 读寄存器
else
{
    // 循环将每个寄存器的数据从寄存器中读取到缓冲区。
    while (usNRegs > 0)
    {
        *pucRegBuffer++ = (unsigned
char)(REG_HOLD_BUF[usRegIndex] >> 8);
        *pucRegBuffer++ = (unsigned char)(REG_HOLD_BUF[usRegIndex]
& 0xFF);
        usRegIndex++;
        usNRegs--;
    }
}

return MB_ENOERR;
}

/**
 * @brief CMD1、5、15 命令处理回调函数
 *
 * 该函数用于处理 Modbus 协议中的 CMD1、CMD5 和 CMD15 命令。
 * 它主要负责读取或写入寄存器中的位数据。
 *
 * @param pucRegBuffer 指向寄存器缓冲区的指针，用于读取或写入数据。
 * @param usAddress 要操作的寄存器起始地址。
 */
```



```
* @param usNCoils 要操作的位数。
* @param eMode 操作模式，可以是读或写。
*
* @return 返回操作结果，如果成功则返回 MB_ENOERR，否则返回相应的错误代码。
*/
eMBCoilsCB(UCHAR *pucRegBuffer, USHORT usAddress,
USHORT usNCoils, eMBRegisterMode eMode)
{
    // 计算寄存器索引
    USHORT usRegIndex = usAddress - 1;
    // 用于位操作的变量
    UCHAR ucBits = 0;
    // 用于存储位状态的变量
    UCHAR ucState = 0;
    // 用于循环操作的变量
    UCHAR ucLoops = 0;

    // 非法检测：检查访问范围是否超出寄存器缓冲区大小。
    if ((usRegIndex + usNCoils) > REG_COILS_SIZE)
    {
        return MB_ENOREG;
    }

    // 根据操作模式执行相应的操作
    if (eMode == MB_REG_WRITE)
    {
        // 计算需要循环的次数
        ucLoops = (usNCoils - 1) / 8 + 1;
        // 写操作
        while (ucLoops != 0)
        {
            // 获取当前寄存器的状态
            ucState = *pucRegBuffer++;
            // 位操作
        }
    }
}
```

```
        ucBits = 0;
        // 遍历每个位
        while (usNCoils != 0 && ucBits < 8)
        {
            // 将状态写入寄存器缓冲区
            REG_COILS_BUF[usRegIndex++] = (ucState >> ucBits) &
0X01;

            // 更新剩余位数
            usNCoils--;
            // 更新位索引
            ucBits++;
        }
        // 更新循环次数
        ucLoops--;
    }
}
else
{
    // 计算需要循环的次数
    ucLoops = (usNCoils - 1) / 8 + 1;
    // 读操作
    while (ucLoops != 0)
    {
        // 初始化状态变量
        ucState = 0;
        // 位操作
        ucBits = 0;
        // 遍历每个位
        while (usNCoils != 0 && ucBits < 8)
        {
            // 根据寄存器缓冲区的状态更新状态变量
            if (REG_COILS_BUF[usRegIndex])
            {
                ucState |= (1 << ucBits);
            }
        }
    }
}
```

```
        }
        // 更新剩余位数
        usNCoils--;
        // 更新寄存器索引
        usRegIndex++;
        // 更新位索引
        ucBits++;
    }
    // 将状态写入寄存器缓冲区
    *pucRegBuffer++ = ucState;
    // 更新循环次数
    ucLoops--;
}
}

return MB_ENOERR;
}

/**
 * @brief CMD2 命令处理回调函数
 *
 * 该函数用于处理 MODBUS 协议中的 CMD2 命令，主要负责读取离散输入寄存器的值。
 *
 * @param pucRegBuffer 指向存放寄存器数据的缓冲区。
 * @param usAddress 寄存器的起始地址。
 * @param usNDiscrete 要读取的离散输入寄存器的数量。
 *
 * @return 返回执行结果，如果成功则返回 MB_ENOERR，否则返回相应的错误代码。
 */
eMBCErrorCode eMBRegDiscreteCB(UCHAR *pucRegBuffer, USHORT usAddress,
USHORT usNDiscrete)
{
    // 计算寄存器索引，从 0 开始
    USHORT usRegIndex = usAddress - 1;
```

```
// 用于处理位操作的变量
UCHAR ucBits = 0;
// 用于存储当前寄存器状态的变量
UCHAR ucState = 0;
// 用于控制循环次数的变量
UCHAR ucLoops = 0;

// 非法检测：检查访问范围是否超出寄存器缓冲区大小
if ((usRegIndex + usNDiscrete) > REG_DISC_SIZE)
{
    return MB_ENOREG;
}

// 计算需要循环的次数，每次循环处理最多 8 个离散输入
ucLoops = (usNDiscrete - 1) / 8 + 1;
// 循环读取离散输入寄存器的值
while (ucLoops != 0)
{
    ucState = 0;
    ucBits = 0;
    // 读取每个寄存器的值，并将其状态更新到 ucState 变量中
    while (usNDiscrete != 0 && ucBits < 8)
    {
        if (REG_DISC_BUF[usRegIndex])
        {
            ucState |= (1 << ucBits);
        }
        usNDiscrete--;
        usRegIndex++;
        ucBits++;
    }
    // 将读取到的状态值存入缓冲区中
    *pucRegBuffer++ = ucState;
    ucLoops--;
```

```
}

// 模拟离散量输入被改变，这里简单地将每个寄存器的状态取反
for (usRegIndex = 0; usRegIndex < REG_DISC_SIZE; usRegIndex++)
{
    REG_DISC_BUF[usRegIndex] = !REG_DISC_BUF[usRegIndex];
}

return MB_ENOERR;
}

/**
 * @brief 将 Modbus 库错误码映射为 Modbus 异常码
 *
 * @param error Modbus 库错误码 (eMBCErrorCode)
 * @return 对应的 Modbus 异常码
 */
uint8_t mapErrorToException(eMBCErrorCode error)
{
    switch (error)
    {
        case MB_ENOREG: // 非法寄存器地址
            return 0x02; // 非法数据地址
        case MB_EINVAL: // 非法参数
            return 0x03; // 非法数据值
        case MB_ENORES: // 资源不足
            return 0x04; // 从站设备故障
        case MB_ETIMEOUT: // 超时
            return 0x06; // 从站设备忙
        case MB_EPORTERR: // 端口错误
            return 0x04; // 从站设备故障
        case MB_ENOERR: // 无错误
            return 0x00; // 无异常
        default: // 未知错误
    }
```

```
        return 0x04;    // 从站设备故障
    }
}
```

### 5.5.2 port.h

```
/*
 * FreeModbus Library: BARE Port
 * Copyright (C) 2006 Christian Walter <wolti@sil.at>
 *
 * This library is free software; you can redistribute it and/or
 * modify it under the terms of the GNU Lesser General Public
 * License as published by the Free Software Foundation; either
 * version 2.1 of the License, or (at your option) any later version.
 *
 * This library is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
 * Lesser General Public License for more details.
 *
 * You should have received a copy of the GNU Lesser General Public
 * License along with this library; if not, write to the Free Software
 * Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-
1301 USA
 *
 * File: $Id$
 */

#ifndef _PORT_H
#define _PORT_H

#include <assert.h>
#include <inttypes.h>
#include "usart.h"
```

```
#include "tim.h"

#define INLINE                inline
#define PR_BEGIN_EXTERN_C    extern "C" {
#define PR_END_EXTERN_C      }

#define ENTER_CRITICAL_SECTION( )
#define EXIT_CRITICAL_SECTION( )

typedef uint8_t  BOOL;

typedef unsigned char  UCHAR;
typedef char  CHAR;

typedef uint16_t  USHORT;
typedef int16_t  SHORT;

typedef uint32_t  ULONG;
typedef int32_t  LONG;

// 定义十路输入寄存器的大小
#define REG_INPUT_SIZE 10
// 声明输入寄存器缓冲区，用于存储十路输入寄存器的值
extern uint16_t REG_INPUT_BUF[REG_INPUT_SIZE];

// 定义十路保持寄存器的大小
#define REG_HOLD_SIZE 10
// 声明保持寄存器缓冲区，用于存储十路保持寄存器的值
extern uint16_t REG_HOLD_BUF[REG_HOLD_SIZE];

// 定义十路线圈的大小
#define REG_COILS_SIZE 10
// 声明线圈缓冲区，并初始化，用于存储十路线圈的状态
extern uint8_t REG_COILS_BUF[REG_COILS_SIZE];
```

```
// 定义十路离散量的大小
#define REG_DISC_SIZE 10
// 声明离散量缓冲区，并初始化，用于存储十路离散量的状态
extern uint8_t REG_DISC_BUF[REG_DISC_SIZE];

#ifndef TRUE
#define TRUE 1
#endif

#ifndef FALSE
#define FALSE 0
#endif

#endif
```

## 5.6 去除断言依赖

此时编译代码会报错：

```
Linking...
PP01_ElectricMachineControl\PP01_ElectricMachineControl.axf: Error: L6218E: Undefined symbol __aeabi_assert (referred from mbutils.o).
Not enough information to list image symbols.
Not enough information to list load addresses in the image map.
Finished: 2 information, 0 warning and 1 error messages.
"PP01_ElectricMachineControl\PP01_ElectricMachineControl.axf" - 1 Error(s), 1 Warning(s).
Target not created.
Build Time Elapsed: 00:00:06
```

进入 keil 软件中，在 C/C++ 栏中添加-DNDEBUG 字段，去除断言依赖：



